In this assignment I use AI, to help me guide throughout the project step by step and help me solve problems along the way if it isn't working. First, I started with:

"How to get started and tell me step by step, so I can understand as I go."

Let me break this down into a clear implementation plan + code structure, so you can actually finish it without getting stuck.

Step 1: SQLite Setup

Database file

totally_not_my_privateKeys.db

Table schema

```
CREATE TABLE IF NOT EXISTS keys(
  kid INTEGER PRIMARY KEY AUTOINCREMENT,
  key BLOB NOT NULL,
  exp INTEGER NOT NULL
);
```

✅ **Python Example (SQLite connection)**

```
import sqlite3

DB_NAME = "totally_not_my_privateKeys.db"

def get_db():
  return sqlite3.connect(DB_NAME)
```

🔐 **Step 2: Key Generation + Storage**

You'll:

- Generate RSA keys
- Serialize them (PEM format)
- Store in DB with expiration timestamp

Generate + Save Keys

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
```

```
import time

def generate_key(expiration_offset_seconds):
  private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
  )

  pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
  )

  exp_time = int(time.time()) + expiration_offset_seconds

  return pem, exp_time
```

Insert into DB (⚠️ SQL Injection Safe)

```
def insert_key(pem, exp):
  conn = get_db()
  cursor = conn.cursor()

  cursor.execute(
    "INSERT INTO keys (key, exp) VALUES (?, ?)",
    (pem, exp)   # ✅ parameterized query
  )

  conn.commit()
  conn.close()
```

🔴 IMPORTANT (SQL Injection Prevention)

Bad ❌:

```
cursor.execute(f"SELECT * FROM keys WHERE exp > {time.time()}")
```

Good ✅:

```
cursor.execute("SELECT * FROM keys WHERE exp > ?", (time.time(),))
```

⏳ Step 3: Preload Keys

You MUST have:

- 1 expired key
- 1 valid key

```
def seed_keys():
  expired_key = generate_key(-10)    # already expired
  valid_key = generate_key(3600)     # valid for 1 hour

  insert_key(*expired_key)
  insert_key(*valid_key)
```

Call this when server starts.

🔑 Step 4: POST `/auth`

Logic:

- If `?expired=true` → use expired key
- Else → use valid key
- Sign JWT

Fetch Key

```
def get_key(expired=False):
  conn = get_db()
  cursor = conn.cursor()

  if expired:
    cursor.execute("SELECT * FROM keys WHERE exp <= ? LIMIT 1", (time.time(),))
  else:
    cursor.execute("SELECT * FROM keys WHERE exp > ? LIMIT 1", (time.time(),))

  row = cursor.fetchone()
  conn.close()
  return row
```

Deserialize + Sign JWT

```
import jwt

def sign_jwt(private_key_pem, kid):
  private_key = serialization.load_pem_private_key(
    private_key_pem,
    password=None
  )
```

```
    payload = {
        "sub": "userABC",
        "iat": int(time.time())
    }

    token = jwt.encode(
        payload,
        private_key,
        algorithm="RS256",
        headers={"kid": str(kid)}
    )

    return token
```

Flask Endpoint

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/auth", methods=["POST"])
def auth():
    expired = "expired" in request.args

    row = get_key(expired)
    if not row:
        return jsonify({"error": "No key found"}), 500

    kid, key_pem, exp = row
    token = sign_jwt(key_pem, kid)

    return jsonify({"token": token})
```

🌐 Step 5: JWKS Endpoint

Goal:

Return **public keys only** in JWKS format

Convert to JWKS

```
from cryptography.hazmat.primitives import serialization
import base64
```

```
def pem_to_jwk(private_key_pem, kid):
  private_key = serialization.load_pem_private_key(
     private_key_pem,
     password=None
  )

  public_key = private_key.public_key()
  numbers = public_key.public_numbers()

  def b64(n):
     return base64.urlsafe_b64encode(n.to_bytes((n.bit_length()+7)//8, 'big')).rstrip(b'=').decode()

  return {
     "kty": "RSA",
     "kid": str(kid),
     "use": "sig",
     "alg": "RS256",
     "n": b64(numbers.n),
     "e": b64(numbers.e)
  }
```

Endpoint

```
@app.route("/.well-known/jwks.json", methods=["GET"])
def jwks():
  conn = get_db()
  cursor = conn.cursor()

  cursor.execute("SELECT * FROM keys WHERE exp > ?", (time.time(),))
  rows = cursor.fetchall()
  conn.close()

  keys = [pem_to_jwk(row[1], row[0]) for row in rows]

  return jsonify({"keys": keys})
```

Then I had problem running the program: so i asked “how to run the server”

Here’s the **step-by-step way to run your JWKS server**. You can even copy this into your README under a “How to Run” section:

How to Run the JWKS Server

1️⃣ Make sure Python is installed

- Python 3.8 or higher is recommended

python --version

2️⃣ Install dependencies

If you haven't installed the required libraries yet:

pip install flask cryptography pyjwt

Or if you have a `requirements.txt` file:

pip install -r requirements.txt

3️⃣ Start the server

From the project folder (where `app.py` is located):

python app.py

You should see output like:

* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

4️⃣ Test the endpoints

Generate a JWT (valid key):
curl -X POST http://127.0.0.1:5000/auth

Generate a JWT (expired key):
curl -X POST "http://127.0.0.1:5000/auth?expired=true"

Get JWKS (public keys):
curl http://127.0.0.1:5000/.well-known/jwks.json

Sometimes, I have to showcase the error messages, so ai will know where I went wrong:

"Internal Server Error The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application. after i wrote: http://127.0.0.1:8080/.well-known/jwks.json"

Ah! That **“Internal Server Error” (500)** happens when your Flask route crashes—most likely because the database or keys aren’t set up correctly yet. Let’s troubleshoot carefully.

🔹 **Step 1: Check the Flask console**

When you access `/.well-known/jwks.json`, Flask should print a **Python traceback** in the terminal. Look for:

Traceback (most recent call last):
 ...
sqlite3.OperationalError: no such table: keys

or something like:

TypeError: object expected bytes, got str

The exact error will tell us **why it failed**.

🔹 **Step 2: Common Causes for JWKS Endpoint Crash**

1. **Database file missing or table not created**
    - The table `keys` must exist:
2. CREATE TABLE IF NOT EXISTS keys(
   kid INTEGER PRIMARY KEY AUTOINCREMENT,
   key BLOB NOT NULL,
   exp INTEGER NOT NULL
);
    - If this was not run, queries will fail → 500 error.
3. **No keys in the database**
    - If `/auth` or JWKS tries to read keys and DB is empty → crash
    - You need to **seed at least one expired and one valid key**.
4. **Key deserialization error**
    - Keys must be **stored as PEM bytes** in the database.
    - Reading a string instead of bytes will throw an error when converting to a JWK.

🔹 **Step 3: Quick Fix – Seed Keys on Startup**

Here’s a snippet to put at the top of your `app.py`:

import time

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import sqlite3
import os

DB_NAME = "totally_not_my_privateKeys.db"

def get_db():
  return sqlite3.connect(DB_NAME)

def create_table():
  conn = get_db()
  conn.execute("""
  CREATE TABLE IF NOT EXISTS keys(
    kid INTEGER PRIMARY KEY AUTOINCREMENT,
    key BLOB NOT NULL,
    exp INTEGER NOT NULL
  )
  """)
  conn.commit()
  conn.close()

def generate_key(expiration_offset_seconds):
  private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
  )
  pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
  )
  exp_time = int(time.time()) + expiration_offset_seconds
  return pem, exp_time

def insert_key(pem, exp):
  conn = get_db()
  conn.execute("INSERT INTO keys (key, exp) VALUES (?, ?)", (pem, exp))
  conn.commit()
  conn.close()

def seed_keys():
  # Only seed if DB is empty
  conn = get_db()
  cursor = conn.cursor()
  cursor.execute("SELECT COUNT(*) FROM keys")
  count = cursor.fetchone()[0]
  conn.close()
  if count == 0:
```

```
    expired_key = generate_key(-10)   # already expired
    valid_key = generate_key(3600)    # expires in 1 hour
    insert_key(*expired_key)
    insert_key(*valid_key)
    print("✅ Keys seeded into database")

# Run on startup
create_table()
seed_keys()
```

🔹 **Step 4: Restart Server**

After adding the seeding code:

py app.py

Now try:

http://127.0.0.1:8080/.well-known/jwks.json

Also this “when i click the link: http://127.0.0.1:8080/ it gives me this: Not Found The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.”

Ah! That “Not Found” page at `http://127.0.0.1:8080/` is **expected** with your current `app.py`.

Here’s why:

🔹 **How Flask routes work**

- In your `app.py`, you only defined **two routes**:

Endpoint	Methods	Purpose
`/auth`	GET, POST	Issue JWTs
`/.well-known/jwks.json`	GET	Serve JWKS

- You **did not define a route for `/`**, so Flask returns **404 Not Found** if you try to open the base URL.

🔹 **Options to fix / test**

1. **Access the correct endpoints directly**:

- JWKS: `http://127.0.0.1:8080/.well-known/jwks.json`
- JWT auth: `http://127.0.0.1:8080/auth`
- Expired JWT: `http://127.0.0.1:8080/auth?expired=true`

2. **Optional:** Add a simple home page (`/`) so it doesn't show 404:

```
@app.route("/", methods=["GET"])
def home():
  return "<h2>JWKS Server is running. Use /auth or /.well-known/jwks.json</h2>"
```

- Now if you open `http://127.0.0.1:8080/`, you'll see a friendly message instead of "Not Found."